Лекция 6. Алгоритм Диница и идеи масштабирования потока.

Вступление

Мы рассмотрели два подхода: алгоритм Форда-Фалкерсона и его улучшенную версию (Эдмондса-Карпа), где мы ищем кратчайший путь (BFS). Мы выяснили, что если искать не любой дополняющий путь, а путь с минимальным количеством рёбер, то количество итераций ограничено $O(N \cdot M)$, а общая сложность составляет $O(N \cdot M^2)$. Однако у этого подхода есть неэффективность: мы каждый раз запускаем полный BFS на весь граф, находим всего один путь, пускаем по нему поток и выбрасываем всю остальную информацию. Далее рассмотрим, как использовать результаты BFS более эффективно.

1. Слоистая сеть (Level Graph) и блокирующий поток

Главная идея алгоритма Диница (Dinic's Algorithm) заключается в том, чтобы после каждого BFS не останавливаться, а найти сразу все кратчайшие пути и пропустить по ним поток, пока они не заблокируются.

Шаг 1. Построение слоистой сети

Мы запускаем BFS из истока s и для каждой вершины v вычисляем расстояние dist[v] (минимальное количество рёбер от s).

Теперь мы строим новый граф – **слоистую сеть**. Мы оставляем только те рёбра (в остаточной сети), которые ведут из слоя dist[u] в слой dist[v] = dist[u] + 1. То есть, рёбра, которые могут лежать на каком-то кратчайшем пути из s в t.

Шаг 2. Удаление тупиков (опционально)

В слоистой сети есть вершины, из которых нельзя дойти до t (тупики). Их можно удалить каскадно: если из вершины не выходит ни одного ребра в следующий слой (кроме t), она бесполезна, и все входящие в неё рёбра также можно удалить. Это упрощает граф.

Шаг 3. Поиск блокирующего потока

В слоистой сети любой путь из s в t является кратчайшим. Мы будем искать такие пути и насыщать их, пока они существуют.

Псевдокод (базовый уровень):

Алгоритм Диница (базовый уровень с удалением тупиков):
1. Инициализируем поток f = 0.
2. Пока истина:
3. // Фаза: строим слоистую сеть
4. Запускаем BFS из s в остаточной графе, вычисляем dist[].
5. Если t не достижима (dist[t] = ∞): выход из цикла.
6. Строим слоистую сеть G_L из рёбер (u -> v), где остаточная пропускная способность > 0 и dist[v] = dist[u] + 1.
7.
8. // Поиск блокирующего потока в G_L

```
Пока в G_L есть путь из s в t:
    // Находим любой путь P в G_L (можно простым DFS, т.к. граф ацикличен)
    delta = min(остаточная пропускная способность рёбер на P)
    Увеличиваем поток на delta вдоль P.
    // Обновляем G_L (удаляем насыщенные рёбра и тупики)
    Для каждого ребра e на P:
        Пропускная способность e уменьшается на delta.
        Если e стало насыщено (capacity = 0):
            Удаляем e из G_L.
    // Каскадно удаляем тупики
    Пока есть вершина v (v != s, v != t) с исходящей степенью 0 в G_L:
        Удаляем все входящие в v рёбра из G_L.
        Удаляем v.
Конец алгоритма
```

Анализ эффективности:

Каждый раз, когда мы находим путь (внутренний цикл while), мы гарантированно удаляем как минимум одно ребро (то, на котором достигли минимума). Так как рёбер в графе M, внутренний цикл выполнится не более O(M) раз за фазу. Внешний цикл выполняется до тех пор, пока расстояние до t увеличивается. Расстояние не может превысить N, поэтому фаз не более O(N). Итоговая сложность: $\boldsymbol{O(N \cdot M)}$ для поиска блокирующего потока наивным способом.

2. Оптимизация поиска блокирующего потока (DFS с зачисткой)

Вместо того чтобы явно удалять тупики, можно сделать поиск пути «умным» DFS, который сам будет выкидывать бесперспективные рёбра.

Идея: запускаем DFS из s. Если мы зашли в вершину и не смогли найти путь до t, значит эта вершина – тупик в текущей слоистой сети. Мы просто удаляем ребро, по которому пришли, и больше никогда в неё не пойдём.

Псевдокод (продвинутый уровень):

```
// Рекурсивная функция для пуска потока из вершины u
Функция dfs(u, flow):
  Если u == t:
    Возврат flow

  Для каждого ребра e = (u -> v) из списка смежности (в слоистой сети G_L):
    Если остаточная способность e > 0:
      pushed = dfs(v, min(flow, capacity[e]))
      Если pushed > 0:
        // Уменьшаем способность прямого ребра, увеличиваем обратное
        capacity[e] -= pushed
        capacity[обратное_ребро] += pushed
```

Возврат pushed
Иначе:
// Мы дошли до тупика. Удаляем это ребро из G_L (или просто помечаем как бесполезное на время этой фазы).
Удалить e из списка смежности u.

// Ни одно ребро не дало результата
Возврат 0

// Основная часть алгоритма (внутри фазы)
Пока истина:
pushed = dfs(s, INF)
Если pushed == 0:
break // Блокирующий поток найден
total_flow += pushed

Этот подход работает, потому что каждое ребро удаляется не более одного раза за фазу.

3. Масштабирование и структуры данных (Link-Cut Tree)

Мы получили алгоритм сложности $O(N^2 \cdot M)$. Это лучше, чем $O(N \cdot M^2)$ у Эдмондса-Карпа при плотных графах. Но можно ли быстрее? Да, если объединить идею Диница с **масштабированием пропускной способности** (capacity scaling) или использовать продвинутые структуры данных, чтобы находить пути не по одному, а сразу «перепрыгивать».

Проблема: мы каждый раз ищем путь, доходим до тупика, откатываемся. Хотелось бы сохранять уже найденные фрагменты путей и не начинать поиск заново от s, а продолжать от места остановки.

Идея «леса путей».

Представьте, что мы строим дерево (или лес) из уже найденных и частично насыщенных путей. У каждой вершины может быть не более одного исходящего «красного» (помеченного) ребра – того, по которому мы пытаемся пустить поток.

- Мы начинаем путь от s.
- Идём по «красным» рёбрам до корня текущего дерева (вершины, у которой нет исходящего красного ребра).
- Если корень – не t, мы берём любое другое ребро из исходной слоистой сети, ведущее из этой вершины, и «приклеиваем» его к нашему пути (делаем его красным).
- Так мы рано или поздно доходим до t.

Когда путь от s до t найден (это набор красных рёбер), нам нужно:

1. Найти на этом пути ребро с минимальной пропускной способностью (delta).
2. Вычесть delta из всех рёбер пути.

3. Удалить рёбра, ставшие нулевыми (они «разрезают» наш лес).

Чтобы делать это эффективно (за логарифмическое время), нужна структура данных, которая умеет работать с путями в дереве: искать минимум на пути, прибавлять число ко всем рёбрам пути, разрезать и склеивать деревья. Это умеет делать **Link-Cut Tree** (дерево ссылок и отсечений).

Псевдокод (идея с LCT):

```
// Используем Link-Cut Tree, где каждая вершина хранит:
// - min_capacity на пути до корня
// - lazy-прибавку

Пока фаза не закончена:
   v = s
   Пока истина:
      // Ищем корень дерева, в котором лежит v
      root = find_root(v)
      Если root == t:
         // Нашли путь до t
         delta = query_min(s, t) // минимум на пути s-t
         если delta == 0: break
         add_to_path(s, t, -delta) // вычитаем дельту
         // Находим ребро, где capacity стало 0, и разрезаем дерево
         cut_edge = find_zero_edge(s, t)
         cut(cut_edge)
         // Продолжаем поиск от s или от места разреза
         v = s
      Иначе:
         // Надо удлинить путь из корня
         // Берём любое ребро из root в u (по слоистой сети), где u в следующем
слое
         e = get_any_edge(root)
         Если такого ребра нет:
            // Вершина root - тупик
            // Удаляем все входящие в root рёбра (разрезаем лес)
            cut_incoming(root)
            // Откатываемся
            v = parent[root] ?? s
         Иначе:
            // Делаем link между root и u
            link(root, u, capacity[e])
            v = u
```

С использованием Link-Cut Tree все операции (link, cut, min, add) занимают $O(\log N)$. Количество операций по-прежнему линейно относительно количества рёбер и фаз, поэтому общая сложность снижается до $\boldsymbol{O(N \cdot M \cdot \log N)}$.

4. Альтернативный подход – проталкивание из вершины (Push from vertex)

Вернёмся к идее слоистой сети. Вместо того чтобы гонять DFS вдоль путей, можно попробовать «разгружать» целые вершины, а не рёбра.

В чём идея? Поток через вершину v ограничен минимумом из суммы входящих и суммы исходящих пропускных способностей.
max_flow_through_v = min(sum(in_capacity[v]), sum(out_capacity[v]))

План:

1. Находим в слоистой сети вершину v (не s и не t) с минимальным значением max_flow_through_v. Пусть это значение равно delta.
2. Мы хотим пропустить delta единиц потока через эту вершину.
3. **Этап 1 (сток):** «проталкиваем» delta единиц потока от v к стоку t. Используем жадный алгоритм: проходим по исходящим рёбрам и насыщаем их, пока не протолкнём всю delta. При этом некоторые рёбра станут нулевыми и удаляются, а в промежуточных вершинах может накопиться излишек потока, который мы рекурсивно проталкиваем дальше.
4. **Этап 2 (исток):** теперь нам нужно доставить эти delta единиц потока от истока s к вершине v. Делаем то же самое, но по обратным рёбрам (или по входящим в v), забирая поток из входящих дуг.
5. После успешного проталкивания, потенциал вершины v исчерпан, и её можно удалить из рассмотрения в текущей фазе.

Почему это работает быстро?

- Каждый раз, когда мы проталкиваем поток из вершины, мы гарантированно удаляем как минимум одно ребро (то, которое стало насыщенным при проталкивании), кроме, возможно, последнего ребра в проталкивании, которое может остаться не полностью насыщенным.
- Следовательно, количество операций проталкивания ограничено количеством рёбер M.
- Так как фаза (построение слоистой сети) повторяется не более N раз, общая сложность алгоритма – $\boldsymbol{O(N \cdot M)}$, но с меньшей константой, чем у DFS версии, хотя и без использования сложных структур данных.
- Этот метод лежит в основе алгоритма Малхотры-Кумара-Махешвари (MKM), который считается одним из самых быстрых для определённых классов графов.